

Sliding Window

Complete Guide

1. What is it?

Sliding window is a technique where you maintain a **contiguous range** [left, right] over an array or string, expanding it (moving right) and shrinking it (moving left) based on some condition, instead of re-scanning every possible subarray/substring from scratch.

It's really a specialized case of the same-direction two-pointer pattern, applied specifically to **contiguous ranges** rather than arbitrary index pairs.

2. Why is it used?

- Avoids re-computing a subarray's sum/count/frequency from scratch for every window ($O(n^2)$ or $O(n^3)$ brute force $\rightarrow O(n)$)
- The window's state (sum, character counts, distinct count) is updated incrementally as it slides — add what enters, remove what leaves
- Works because of monotonicity: as the window grows, the property you're tracking changes predictably, so you never need to re-examine discarded parts

3. Where is it used?

- Longest/shortest substring with a condition (no repeating characters, at most K distinct, contains all characters of another string)
- Maximum/minimum sum subarray of size K, or with a sum constraint
- Longest subarray with at most K zeros/replacements (flip problems)
- Anagram / permutation detection in a string
- Minimum window substring (shortest window containing all of a target's characters)
- Fixed-size window problems (moving average, max in sliding window of size K)
- Count subarrays satisfying an exact condition (often via "at most K" – "at most K-1" trick)

4. How to identify a Sliding Window problem (pattern triggers)

Signal in the problem statement	What it suggests
"subarray" / "substring" (contiguous!)	Strong signal — sliding window operates only on contiguous ranges
"longest/shortest ... satisfying condition"	Variable-size window
"of size K" / "window of size K"	Fixed-size window
"at most K distinct", "no more than K", "at least K"	Variable window with a shrink condition
"contains all characters of", "anagram of", "permutation of"	Frequency-map window (often fixed size for permutation, variable for min window)
"sum equals/at least/at most target" over a subarray	Sum-tracking window (only works cleanly with non-negative numbers)
Brute force would be $O(n^2)$ checking every subarray	Sliding window collapses it to $O(n)$

Paraphrase test: If the problem can be rephrased as "find the longest/shortest/count of contiguous ranges where property P holds" — it's sliding window. If "contiguous" doesn't apply (any subset, not just adjacent elements), it's NOT sliding window — look at subsets/DP/backtracking instead.

Important caveat: sliding window with a simple expand/shrink works cleanly when the tracked quantity is **monotonic** as the window grows — e.g., sum of non-negative numbers only increases as you add elements. If negative numbers are allowed, growing the window doesn't monotonically increase the sum, so plain sliding window breaks — look at prefix sums + hashmap instead.

5. Can you tell from constraints alone?

Yes:

- n up to 10^5 – 10^6 with an expected $O(n)$ or $O(n \log n)$ solution, and the problem mentions "subarray"/"substring" → sliding window is a prime candidate over brute force.
- If all values are non-negative (sums, counts, positive array elements) → sum/count-based sliding window is safe.
- If negative numbers are present and the problem is about **sum**, sliding window alone won't work — that's usually prefix sum + hashmap instead.
- If the alphabet/character set is small and bounded (e.g., lowercase letters only) → frequency-array window (size 26) instead of a hashmap, for $O(1)$ per-character updates.

6. Intuition / Rationale

The window is safe to slide because of a **monotonic invariant**, exactly like two pointers: once the window's property becomes invalid (e.g., sum too large, too many distinct characters), shrinking from the left is the *only* useful move — you never need to re-check a left boundary you've already advanced past, because any window starting there was already accounted for as valid or invalid.

Each element is added to the window exactly once (when `right` passes it) and removed exactly once (when `left` passes it), giving $O(n)$ total work instead of $O(n)$ work repeated for $O(n)$ different window start points.

7. Time & Space Complexity (generic)

- **Time:** $O(n)$ — each pointer moves forward at most n times total
- **Space:** $O(1)$ for simple sum tracking, $O(k)$ or $O(26)/O(128)$ for a frequency map/array (k = distinct elements tracked)

8. Types of Sliding Window (with Java templates)

Type A — Fixed-Size Window

Used for: max sum subarray of size K , moving average, max/min in window of size K (with a deque).

```
public int maxSumFixedWindow(int[] nums, int k) {
    int windowSum = 0;
    for (int i = 0; i < k; i++) windowSum += nums[i]; // build initial window
    int maxSum = windowSum;

    for (int right = k; right < nums.length; right++) {
        windowSum += nums[right] - nums[right - k]; // slide: add new, remove oldest
        maxSum = Math.max(maxSum, windowSum);
    }
    return maxSum;
}
```

Type B — Variable-Size Window (shrink while invalid)

Used for: longest subarray with sum $\leq K$, longest substring with at most K distinct characters.

```
public int longestSubarrayAtMostK(int[] nums, int k) {
    int left = 0, sum = 0, maxLen = 0;
    for (int right = 0; right < nums.length; right++) {
        sum += nums[right];
        while (sum > k) { // shrink while window is invalid
            sum -= nums[left];
            left++;
        }
        maxLen = Math.max(maxLen, right - left + 1); // window is valid here
    }
    return maxLen;
}
```

Type C — Variable-Size Window (expand while invalid, shrink while valid — “shortest” variant)

Used for: minimum window substring, smallest subarray with $\text{sum} \geq \text{target}$.

```
public int minSubArrayLen(int target, int[] nums) {
    int left = 0, sum = 0, minLen = Integer.MAX_VALUE;
    for (int right = 0; right < nums.length; right++) {
        sum += nums[right];
        while (sum >= target) { // shrink while still valid, to find the minimum
            minLen = Math.min(minLen, right - left + 1);
            sum -= nums[left];
            left++;
        }
    }
    return minLen == Integer.MAX_VALUE ? 0 : minLen;
}
```

Type D — Frequency Map Window (character/element matching)

Used for: minimum window substring, find all anagrams, permutation in string.

```
public String minWindow(String s, String t) {
    if (s.isEmpty() || t.isEmpty()) return "";
    Map<Character, Integer> need = new HashMap<>();
    for (char c : t.toCharArray()) need.merge(c, 1, Integer::sum);

    Map<Character, Integer> window = new HashMap<>();
    int have = 0, needCount = need.size();
    int left = 0, bestLen = Integer.MAX_VALUE, bestStart = 0;

    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        window.merge(c, 1, Integer::sum);
        if (need.containsKey(c) && window.get(c).intValue() == need.get(c).intValue()) {
            have++; // this character's frequency requirement is now fully satisfied
        }

        while (have == needCount) { // window is valid, try to shrink from the left
            if (right - left + 1 < bestLen) {
                bestLen = right - left + 1;
                bestStart = left;
            }
            char leftChar = s.charAt(left);
            window.put(leftChar, window.get(leftChar) - 1);
            if (need.containsKey(leftChar) && window.get(leftChar) < need.get(leftChar)) {
                have--; // shrinking broke the requirement for this character
            }
            left++;
        }
    }
    return bestLen == Integer.MAX_VALUE ? "" : s.substring(bestStart, bestStart + bestLen);
}
```

Type E — “At Most K” Subtraction Trick (count exact-K subarrays)

Used for: count subarrays with exactly K distinct elements, exactly K odd numbers, etc.

```
public int subarraysWithExactlyKDistinct(int[] nums, int k) {
    return atMostKDistinct(nums, k) - atMostKDistinct(nums, k - 1);
}

private int atMostKDistinct(int[] nums, int k) {
    if (k < 0) return 0;
    Map<Integer, Integer> count = new HashMap<>();
    int left = 0, result = 0;
    for (int right = 0; right < nums.length; right++) {
        count.merge(nums[right], 1, Integer::sum);
        while (count.size() > k) {
            int leftVal = nums[left];
            count.put(leftVal, count.get(leftVal) - 1);
            if (count.get(leftVal) == 0) count.remove(leftVal);
            left++;
        }
        result += right - left + 1; // every window ending at `right` with ≤k distinct counts
    }
    return result;
}
```

9. Comparison of the patterns

Type	Window behavior	Typical trigger words	Complexity
A. Fixed-size	slides one step at a time, constant width	"size K", "moving average"	$O(n)$
B. Variable — longest under a cap	expand right, shrink while invalid	"longest ... at most K"	$O(n)$
C. Variable — shortest meeting a target	expand right, shrink while still valid	"shortest/minimum ... at least"	$O(n)$
D. Frequency map	tracks character/element counts, not just a sum	"anagram", "permutation", "contains all of t"	$O(n)$ ($O(26)$ or $O(k)$ per step)
E. At-most-K subtraction	two calls to Type B variant	"exactly K distinct/odd/etc."	$O(n)$

Key distinguishing question: Is the window always the same width ($\rightarrow$ A), am I maximizing width under a constraint ($\rightarrow$ B), minimizing width while meeting a threshold ($\rightarrow$ C), matching character frequencies ($\rightarrow$ D), or counting subarrays with an EXACT property ($\rightarrow$ E, via the at-most trick)?

10. Quick decision checklist (use this in an interview)

- 1 Is the window a fixed size K? $\rightarrow$ **Type A**
- 2 Am I looking for the longest window that stays valid ($\text{sum} \leq K$, $\leq K$ distinct)? $\rightarrow$ **Type B** (shrink while invalid)
- 3 Am I looking for the shortest window that becomes valid ($\text{sum} \geq \text{target}$, contains all of t)? $\rightarrow$ **Type C** (shrink while valid, to minimize)
- 4 Do I need to track per-character/per-element frequency rather than just a sum/count? $\rightarrow$ **Type D**
- 5 Does the problem ask for an EXACT count/value rather than longest/shortest? $\rightarrow$ **Type E**, compute $\text{atMost}(k) - \text{atMost}(k-1)$
- 6 Are negative numbers involved and the property is sum-based? $\rightarrow$ **Not sliding window** — use prefix sum + hashmap instead

11. Comparison with Two Pointers, Sweep Line, and Greedy

	Two Pointers	Sliding Window	Sweep Line	Greedy
Core idea	Two indices over one array, often converging from ends	A contiguous range [left, right] that expands/shrinks	Process sorted events, track running state	Locally optimal choice each step
Range shape	Any two positions (not necessarily contiguous between them)	Always contiguous — the range itself is the answer	Points/intervals sorted by position	N/A — a decision strategy
Typical output	Pair/triplet, boolean	Length, count, or substring/subarray itself	Count of overlaps, min resources	Optimal count/cost/sequence
Relationship	Sliding window IS a two-pointer pattern, specialized to contiguous ranges	—	Fixed-size sweep-window over event time is conceptually similar	Greedy criterion often decides how the window should shrink/expand

The cleanest mental model: two pointers is the general family (opposite-direction, same-direction, merge, fast/slow). Sliding window is specifically the “same-direction, contiguous range” member of that family, applied whenever the answer must be a *contiguous* piece of the array/string. If contiguity doesn't matter, you're back to general two pointers or a different technique entirely.